

Chapter 4 – The Attention Mechanism and the Transformer Architecture

Advanced Machine Learning for Finance

Giovanni Della Lunga

University of Bologna

April-May 2026

Introduction

Lesson 1 – Classical NLP

Bag-of-words, TF-IDF, cosine similarity: sparse, high-dimensional, no word-order, no semantics beyond co-occurrence counts.

Lesson 2 – Neural Networks, Embeddings, and Tokenization

Perceptron → gradient learning → Word2Vec / GloVe: dense, distributed, encode semantic similarity, but *static* – one vector per word regardless of context.

Lesson 3 – Recursive Neural Network and Sequence-to-sequence problem

RRN → GRU → LSTM

The encoder-decoder architecture for general seq-2-seq applications

Today's question

How do we move from a single fixed vector per word to representations that capture meaning in context, and what is the architecture that makes this possible at scale?

Part I – The Problem

- ▶ From static to contextual embeddings
- ▶ The polysemy problem in finance
- ▶ Early attempts: RNNs and LSTMs

Part II – The Attention Mechanism

- ▶ Intuition: attention as soft lookup
- ▶ Similarity scores and scaled dot-product attention
- ▶ A step-by-step walkthrough
- ▶ Why scale by $\sqrt{d_k}$?
- ▶ Self-attention and why it works

Part II (cont.)

- ▶ Multi-head attention
- ▶ Attention: interpretation and caution

Part III – Positional Encoding

- ▶ Why do we need positional information?
- ▶ Sinusoidal positional encoding

Part IV – The Transformer

- ▶ Full architecture overview
- ▶ Residual connections & layer norm
- ▶ Feed-forward sub-layer
- ▶ Masked attention & the decoder

From Static to Contextual Embeddings

The Polysemy Problem

Example

- ▶ “The bank raised interest rates.”
- ▶ “We walked along the river bank.”

Word2Vec assigns one vector to “bank”. It conflates all senses into a single point in semantic space.

What we need: a representation that depends on the surrounding words – a function

$$f(\text{word}, \text{context}), \quad \text{not just } f(\text{word}).$$

Financial relevance

“spread”, “yield”, “default”, “call”, “exposure” – domain-specific polysemy is pervasive in finance. A model that cannot disambiguate these in context is fundamentally limited for financial NLP.

Early Attempts: RNNs and LSTMs

Recurrent Neural Networks process tokens sequentially, passing a hidden state forward:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b).$$

Each h_t is contextual: it depends on the words seen so far.

Long Short-Term Memory (LSTM) adds gating mechanisms to control information flow and alleviate vanishing gradients.

Key limitations

- ▶ Sequential processing: cannot parallelise across positions – training is slow.
- ▶ Long-range dependencies: despite gates, information from distant positions degrades.

These limitations motivated the search for a fundamentally different architecture.

Contextual Embeddings: The Core Idea

Word2Vec

“bank” $\mapsto e_{\text{bank}}$

same vector always

Contextual Model

“The bank raised rates” $\rightarrow h_{\text{bank}}^{(1)}$

“Along the river bank” $\rightarrow h_{\text{bank}}^{(2)}$

different vectors

A contextual embedding model produces a **different vector** for each occurrence of a word, conditioned on its full sentential context.

The Transformer architecture achieves this via **attention**, without any sequential recurrence.

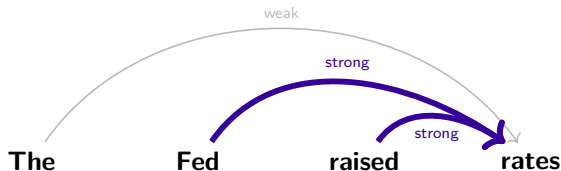
The Attention Mechanism

Attention: The Basic Problem

Before introducing new terminology, start from the practical question:

For each word, which other words should influence its meaning?

In the sentence “The Fed raised rates”, the vector for *rates* should be influenced strongly by *Fed* and *raised*, and only weakly by *The*.

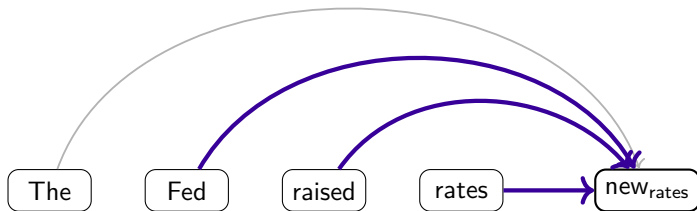


Attention is the mechanism that **learns** these strengths automatically and uses them to build a new, context-dependent vector for each token.

Attention as Soft Selection

A simple way to think about attention is **soft selection**:

- 1 Pick the token we want to update, for example *rates*.
- 2 Compare it with every token in the sentence.
- 3 Give each comparison a numerical score.
- 4 Convert the scores into weights that sum to one.
- 5 Build the new vector as a weighted mixture of information.



The arrows will become numerical weights.

A First Attempt: Similarity-Based Mixing

Simplest idea: use the dot product between embedding vectors as a measure of how much two tokens are related. Tokens whose embeddings are similar (high dot product) should exchange more information; dissimilar tokens should exchange less.

For each token i , compute a new representation z_i as a weighted average of all input embeddings:

$$z_i = \sum_{j=1}^n \alpha_{ij} x_j, \quad \alpha_{ij} = \frac{\exp(x_i^\top x_j)}{\sum_{j'} \exp(x_i^\top x_{j'})}.$$

What is happening here?

- ▶ $x_i^\top x_j$ measures the similarity between tokens i and j in the embedding space.
- ▶ The softmax converts raw similarities into weights that sum to 1.
- ▶ The output z_i is a blend of all input embeddings, with more weight on the most similar ones.

Concrete Example: Raw Similarity Attention

Consider: “The Fed raised rates”. Compute the dot product of each token’s embedding with every other.

	The	Fed	raised	rates
The	3.0	0.1	0.0	0.3
Fed	0.1	3.2	1.8	2.1
raised	0.0	1.8	3.5	2.7
rates	0.3	2.1	2.7	3.4

Raw dot-product scores $s_{ij} = x_i^\top x_j$.

- ▶ Diagonal is large: every word is most similar to itself.
- ▶ “rates” and “raised”: score 2.7 – semantically related words have higher similarity.
- ▶ “The” and “rates”: score 0.3 – function words are distant from content words.

After softmax, the new embedding of “rates” would be a blend dominated by itself and “raised” – already a rough contextualisation.

Introducing the Query, Key, Value Representation

Consider one token, *rates*. It gets a query vector q_{rates} ; every token gets a key vector k_j . The raw relevance score is a scalar product:

$$\alpha_{\text{rates},j} = q_{\text{rates}}^\top k_j.$$

The new vector is then a weighted sum of value vectors:

$$z_{\text{rates}} = \sum_j \alpha_{\text{rates},j} v_j.$$

The words *query*, *key*, and *value* are just names for three roles in the soft lookup.

Query

What the current token is looking for.

For “rates”: what context helps explain me?

Key

What each token advertises about itself.

“Fed” advertises central-bank information.

Value

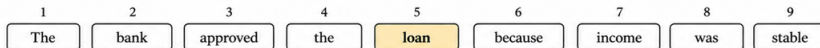
The information that will be passed on if that token is relevant.

The content contributed by “Fed”.

Simple Self-Attention (Without Weight Matrices)

A first step toward the full attention mechanism

Running example (financial): *The bank approved the loan because income was stable.*



We focus on the current token $i = 5$ ("loan") and compute its context-aware representation.

1 Input embeddings

Each token j is represented by an embedding vector $x_j \in \mathbb{R}^d$.

1	The	x_1	$x_j = \begin{bmatrix} x_{j1} \\ x_{j2} \\ \vdots \\ x_{jd} \end{bmatrix}$
2	bank	x_2	
3	approved	x_3	
4	the	x_4	
5	loan	x_5	
6	because	x_6	
7	income	x_7	
8	was	x_8	
9	stable	x_9	

x_j is the embedding of token j .

2 Similarity via scalar product

For current token i , compute relevance score with token j as

$$s_{ij} = x_i^T x_j$$

Token j Raw score s_{ij}

1	The	0.06
2	bank	0.18
3	approved	0.44
4	the	0.02
5	loan	1.20
6	because	0.16
7	income	0.90
8	was	0.03
9	stable	0.16

The scalar product is larger when the two embedding vectors point in similar directions.

3 Normalization with softmax

Convert raw scores to attention weights:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{k=1}^9 \exp(s_{ik})}$$

Token j Attention weight α_{ij}

1	The	0.05
2	bank	0.10
3	approved	0.17
4	the	0.04
5	loan	0.24
6	because	0.08
7	income	0.20
8	was	0.04
9	stable	0.08

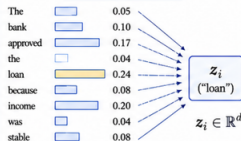
Softmax transforms raw scores into positive attention weights that sum to 1.

$$\sum_{j=1}^9 \alpha_{i,j} = 1$$

4 Context-aware representation

Aggregate all token embeddings using the attention weights:

$$z_i = \sum \alpha_{ij} x_j$$



The new representation is a weighted average of all token embeddings, with larger weights assigned to more relevant tokens.

Why Raw Similarity Is Not Enough

The simple dot-product approach works as a first approximation, but it has three fundamental limitations.

Limitation 1: The same vector plays two roles

The embedding x_i must simultaneously serve as:

- ▶ the *address*: what to match on – “am I relevant to you?”
- ▶ the *content*: what to transmit – “here is my information”

But these are different functions. A word might be highly relevant to another without being semantically similar to it.

Limitation 2: Similarity is symmetric

$x_i^\top x_j = x_j^\top x_i$, but relevance is often asymmetric: “rates” should attend strongly to “raised” to understand what happened, but “raised” may need “Fed” more than “rates”.

Why Raw Similarity Is Not Enough

Limitation 3: No learning

Raw dot products use the embedding space as-is. The model cannot learn task-specific notions of relevance – what matters for sentiment analysis is different from what matters for named entity recognition.

The Solution: Three Separate Learned Projections

The fix is elegant: instead of comparing raw embeddings, project each embedding into three separate spaces using learned weight matrices:

$$q_i = W^Q x_i, \quad k_i = W^K x_i, \quad v_i = W^V x_i.$$

Each projection has a distinct role

- ▶ Q : what this position should look for;
- ▶ K : how this position should be matched by other tokens;
- ▶ V : what information this position contributes to the output.

The key point is that the model learns the comparison space and the content space **separately**.

The Solution: Three Separate Learned Projections

What changed compared to the simple version?

Before

$$s_{ij} = x_i^\top x_j$$

Same vector, dual role, symmetric, fixed.

Now

$$s_{ij} = q_i^\top k_j$$

Different projections, asymmetric, learnable.

And the content transmitted is v_j , not x_j – relevance is decoupled from content.

Scaled Dot-Product Attention

Given matrices $Q, K, V \in \mathbb{R}^{n \times d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V.$$

Step by step:

- 1 Compute all pairwise scalar products: $S = QK^\top$, with $S_{ij} = q_i^\top k_j$.
- 2 Scale by $\sqrt{d_k}$ to prevent softmax saturation.
- 3 Apply softmax row-wise $\rightarrow$ attention weights α_{ij} .
- 4 Output = weighted sum of value vectors.

From Hard Lookup to Soft Lookup

Now that the full formulation has been introduced, we can reinterpret attention as a learnable version of lookup.

Hard lookup: classical dictionary

- ▶ A query matches exactly one key.
- ▶ It returns one associated value.
- ▶ The choice is discrete: the rule is fixed by exact matching.

Soft lookup: attention

- ▶ A token can match all other tokens to varying degrees.
- ▶ It returns a weighted average of their value vectors.
- ▶ The matching rule is learned through W^Q , W^K , and W^V .

Why this is learnable

The scalar products $q_i^\top k_j$ produce the scores, but the vectors q_i , k_j , and v_j are themselves produced by learned matrices. During training, gradient descent adjusts these matrices so that the model learns what to compare and what information to pass on.

Walkthrough: Setting Up the Problem

Let us trace the attention computation on a short sentence:

“The Fed raised rates”

After tokenization and embedding, we have four input vectors $x_1, \dots, x_4 \in \mathbb{R}^d$.
For this walkthrough, suppose $d = d_k = 4$ (toy dimensions).

Step 0: Move from the plain-language roles to their vector form. For every token, the model creates three learned views:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V.$$

- ▶ Each row of Q is the *query* for that token: “what am I looking for?”
- ▶ Each row of K is the *key*: “what do I have that may match?”
- ▶ Each row of V is the *value*: “what information will I pass on if selected?”

Walkthrough: Computing Attention Scores

Step 1: Compute all pairwise scores $S = QK^T$:

$$S = QK^T = \begin{pmatrix} q_1^T k_1 & q_1^T k_2 & q_1^T k_3 & q_1^T k_4 \\ q_2^T k_1 & q_2^T k_2 & q_2^T k_3 & q_2^T k_4 \\ q_3^T k_1 & q_3^T k_2 & q_3^T k_3 & q_3^T k_4 \\ q_4^T k_1 & q_4^T k_2 & q_4^T k_3 & q_4^T k_4 \end{pmatrix}.$$

	The	Fed	raised	rates
The	2.1	0.5	0.3	0.1
Fed	0.4	1.8	1.2	1.5
raised	0.2	1.0	2.0	1.6
rates	0.1	1.4	1.5	2.2

- ▶ Entry (i, j) : how much does token i 's query match token j 's key.
- ▶ Example: “rates” attends strongly to “Fed” (1.4) and “raised” (1.5) – financially meaningful.

Walkthrough: Scaling and Softmax

Step 2: Divide by $\sqrt{d_k}$ (here $\sqrt{4} = 2$):

$$S' = \frac{S}{\sqrt{d_k}}$$

For example, row “rates”: (0.05, 0.70, 0.75, 1.10).

Step 3: Apply softmax row-wise to get attention weights α :

$$\alpha_{ij} = \frac{\exp(S'_{ij})}{\sum_{j'} \exp(S'_{ij'})}.$$

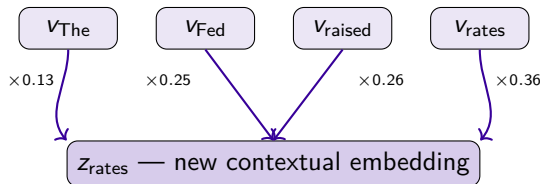
For the row corresponding to “rates”:

$$\alpha_{\text{rates}} = (0.13, 0.25, 0.26, 0.36).$$

Walkthrough: Computing the Output

Step 4: The output for each token is the weighted sum of value vectors:

$$z_i = \sum_{j=1}^n \alpha_{ij} v_j.$$

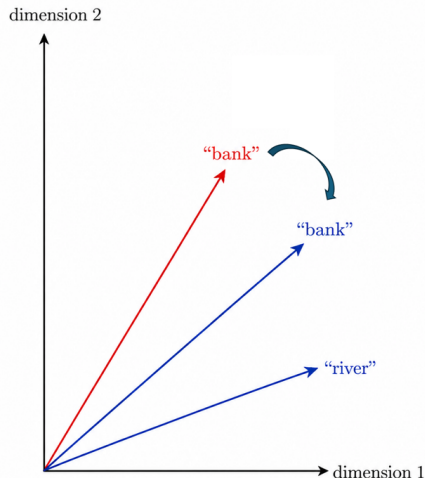


The output z_{rates} is a contextual embedding: it is no longer a static representation of “rates” but one that has been informed by “Fed” and “raised” – capturing the financial meaning of this occurrence.

Why Does Self-Attention Work? – Geometric Intuition

Consider: “I swam across the river to get to the other bank.”

- ▶ The input embeddings of “river” and “bank” occupy a similar region of the semantic space.
- ▶ Their dot product $x_{\text{river}}^\top x_{\text{bank}}$ is large $\rightarrow$ high attention weight.
- ▶ Result: the contextualised embedding of “bank” absorbs a large fraction of the “river” embedding $\rightarrow$ disambiguation.
- ▶ Conversely, function words like “the”, “to”, “across” have small dot products with “bank” $\rightarrow$ low attention weights.



Why Does Self-Attention Work? – The Key Insight

The mechanism in one sentence

Self-attention creates contextual embeddings by letting each token's representation absorb information from semantically related tokens, with the degree of absorption determined by learned similarity in the query-key subspace.

Three cases:

- ① **Strong relationship** (e.g., “river” $\leftrightarrow$ “bank”): large attention weight $\rightarrow$ the contextualised embedding of “bank” is a substantial blend of both input embeddings.
- ② **Weak relationship** (e.g., “the” $\leftrightarrow$ “bank”): small attention weight $\rightarrow$ the contextualised embedding of “the” is nearly identical to its original input embedding.
- ③ **Self-relationship** (e.g., “bank” $\leftrightarrow$ “bank”): typically has high weight $\rightarrow$ the token retains its own identity while being enriched by context.

Why Scale by $\sqrt{d_k}$?

- Without scaling, if d_k is large, the dot products $q^\top k$ grow in magnitude: their variance is approximately d_k for unit-variance entries.
- Large dot products push the softmax into saturated regions where gradients are near zero $\rightarrow$ learning stalls.
- Dividing by $\sqrt{d_k}$ keeps the variance of the logits at approximately 1, ensuring well-behaved gradients.

Why does variance grow with d_k ?

If $q_i, k_i \sim \mathcal{N}(0, 1)$ independently, then

$$q^\top k = \sum_{i=1}^{d_k} q_i k_i$$

is a sum of d_k independent terms, each with variance 1.

$$\text{Var}(q^\top k) = d_k.$$

Dividing by $\sqrt{d_k}$ normalises the variance back to 1.

Self-Attention

When queries, keys, and values all come from the same sequence, we call it **self-attention**. Each token in the sequence attends to every other token, including itself:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V,$$

where $X \in \mathbb{R}^{n \times d}$ is the sequence of input embeddings.

Example: self-attention at work

In “The Federal Reserve raised the interest rate,” the representation of “rate” is informed by “interest”, “Federal”, and “Reserve” – all in a single parallel step.

Computational cost

Self-attention has $\mathcal{O}(n^2)$ complexity in sequence length n because the $QK^\top$ product is $n \times n$. This is the main computational bottleneck for very long documents.

Multi-Head Attention

Instead of a single attention function, run h independent attention “heads” in parallel, each with its own learned projections:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V).$$

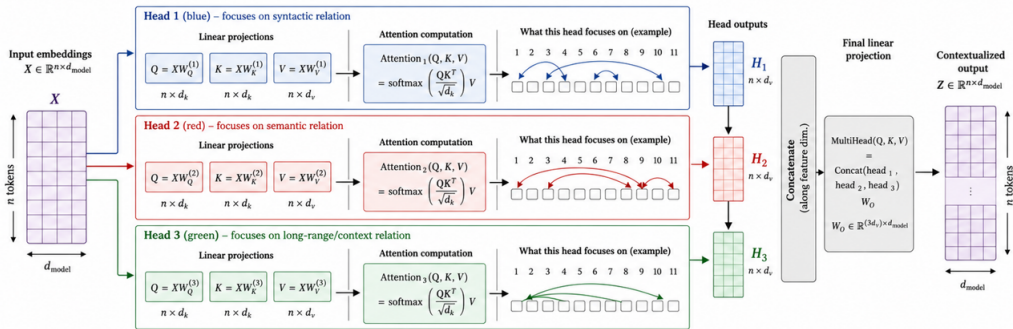
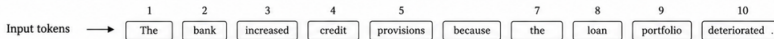
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O.$$

Why multiple heads?

- ▶ Different heads can attend to different relationships: syntactic, semantic, positional, entity-level.
- ▶ With $d_k = d_{\text{model}}/h$, total cost is comparable to single-head attention at full dimensionality.
- ▶ In practice, researchers have observed heads specialising in subject–verb agreement, co-reference, modifier attachment, etc.

Multi-Head Attention

Example sentence (finance): *The bank increased credit provisions because the loan portfolio deteriorated.*



Takeaway: Different heads can learn complementary views of the same sequence, allowing the model to capture multiple relationships in parallel.

Attention Weights: Interpretation and Caution

Attention weights α_{ij} show how much position i “attends to” position j . They are often visualised as heatmaps.

Interpretability caveat

Attention weights are **not** reliable explanations of model decisions.

- ▶ Multiple heads attend differently; no single pattern tells the full story.
- ▶ Weights indicate input mixing, not causal attribution.
- ▶ Research by Jain & Wallace (2019) has shown attention patterns can be rearranged with minimal impact on output.

Use attention visualisations as **illustrative tools**, not as explanations.

Positional Encoding

Why Do We Need Positional Information?

Self-attention is **permutation-equivariant**: it treats the input as a *set* of tokens, not a sequence.

Without positional information, these two sentences would produce identical attention outputs:

“The dog bit the man” vs. “The man bit the dog”

Word order is essential for meaning. RNNs get this for free through sequential processing. Transformers must inject it **explicitly**.

Token Identity vs. Token Position

Each input representation should contain two pieces of information.

Token identity

What is the word?

bank $\mapsto e_{\text{bank}}$

This is represented by the token embedding.

Token position

Where is the word?

pos = 0, 1, 2, ...

This is represented by the positional encoding.

Transformer input

$$z_{\text{pos}} = e_{\text{token}(\text{pos})} + p_{\text{pos}}.$$

The model receives *meaning* plus *position*.

First Attempt: Add the Position Index

A natural first idea is simple:

$$p_{\text{pos}} = \text{pos.}$$

For example:

0, 1, 2, 3, 4, 5, ...

Then we could add the position number to the token embedding.

Why this seems reasonable

Every token receives information about where it occurs in the sequence.

But this solution is too crude

The raw integer value creates several problems for neural networks.

Problem 1: Magnitude

Token embeddings usually contain values of moderate size.
For example, components may be roughly centered around zero:

$$e_{\text{token}} = [0.12, -0.34, 0.08, 0.27, \dots]$$

But raw positions can become large:

$$\text{pos} = 1, 10, 100, 1000, \dots$$

Problem

Large position values may dominate the semantic embedding. The model may struggle to separate:

*what the word **means** from where the word **appears**.*

Problem 2: Normalisation Is Not Enough

A second idea is to normalise positions:

$$p_{\text{pos}} = \frac{\text{pos}}{L},$$

where L is the sequence length. This keeps values between 0 and 1.

New problem

The same absolute position receives different encodings in sequences of different lengths. For example, position 5 has different values if:

$$L = 10 \quad \text{or} \quad L = 100.$$

Desired property

A position encoding should be **consistent across sequences** of different lengths.

What Do We Want from a Positional Code?

A good positional encoding should have several properties.

- ① Each position should have a distinctive code.
- ② Nearby positions should have related codes.
- ③ The code should not explode in magnitude.
- ④ The same position should have the same code across sequences.
- ⑤ The model should be able to reason about relative distances.
- ⑥ The scheme should generalise beyond the sequence lengths seen in training.

Can we design such a positional code from familiar ideas?

A Familiar Positional Code: Binary Counting

Binary already encodes position using multiple components.

$$0 = 000, \quad 1 = 001, \quad 2 = 010, \quad 3 = 011, \quad 4 = 100.$$

Instead of one number, each position gets a vector of bits.

position	0	1	2	3	4	5	6	7
bit ₀	0	1	0	1	0	1	0	1
bit ₁	0	0	1	1	0	0	1	1
bit ₂	0	0	0	0	1	1	1	1

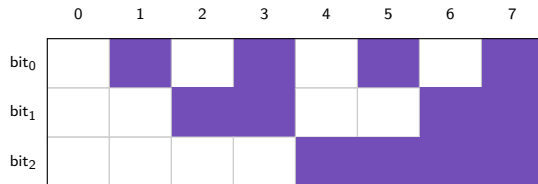
Observation

Binary is a multi-dimensional positional code.

Binary as Multiple Frequencies

Binary bits change at different speeds.

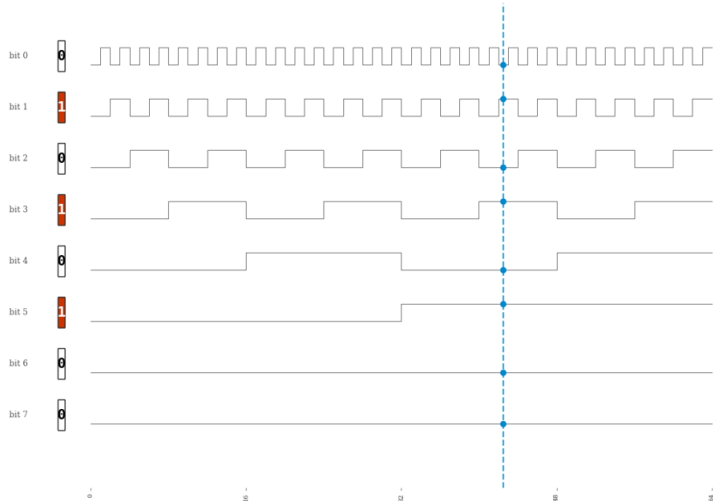
- ▶ The least significant bit flips every step.
- ▶ The next bit flips every two steps.
- ▶ The next bit flips every four steps.
- ▶ Higher-order bits change more slowly.



Key idea

Position can be encoded through signals that vary at **different frequencies**.

Binary as Multiple Frequencies



42
position

Binary Solves Some Problems

Binary has attractive properties.

- ▶ Codes are bounded: values are only 0 or 1.
- ▶ Positions can be uniquely identified.
- ▶ The same position has the same binary representation in any sequence.
- ▶ Different bits capture different positional scales.

Important insight

Binary already contains the essential idea behind sinusoidal positional encoding:

use several frequencies to encode position.

But Binary Is Too Discontinuous

Binary has a serious problem.

$$3 = 011, \quad 4 = 100.$$

The position changes by one step, but all bits change.

Problem

Nearby positions can have very different binary vectors.

This is not ideal for neural networks, because neural networks rely on:

- ▶ continuous representations;
- ▶ smooth transformations;
- ▶ gradients;
- ▶ dot products and distances.

Transition

Can we keep the multi-frequency structure of binary but make it smooth?

From Binary Bits to Smooth Waves

Binary bits are hard oscillators:

0, 1, 0, 1, 0, 1, ...

Sinusoids are smooth oscillators:

$\sin(\omega \cdot pos)$, $\cos(\omega \cdot pos)$.

Binary

- ▶ discrete;
- ▶ discontinuous;
- ▶ abrupt bit flips;
- ▶ hard positional code.

Sinusoidal

- ▶ continuous;
- ▶ smooth;
- ▶ gradual variation;
- ▶ soft positional code.

What Does Frequency Mean?

A sinusoidal positional signal has the form:

$$pos \mapsto \sin(\omega \cdot pos).$$

The frequency ω determines how quickly the signal changes as position increases.

High frequency

- ▶ changes quickly;
- ▶ distinguishes nearby positions;
- ▶ similar to low-order binary bits.

Low frequency

- ▶ changes slowly;
- ▶ captures broad location;
- ▶ similar to high-order binary bits.

Clarification

Frequency is not a property of a word. It is a property of the positional signal.

The Transformer Formula

The original Transformer uses:

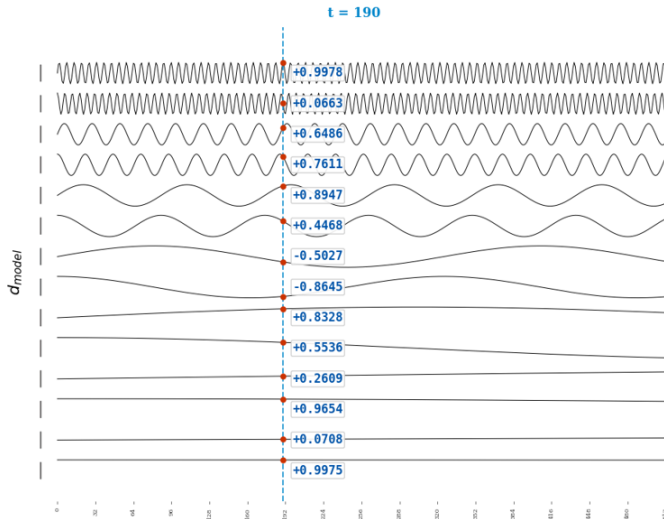
$$PE(pos, 2i) = \sin \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right),$$

$$PE(pos, 2i + 1) = \cos \left(\frac{pos}{10000^{2i/d_{\text{model}}}} \right).$$

Meaning of the symbols

- ▶ pos : token position;
- ▶ i : index of the sine–cosine pair;
- ▶ d_{model} : embedding dimension;
- ▶ even dimensions use sine;
- ▶ odd dimensions use cosine.

The Transformer Formula



$PE(t, 2i) :$

$$\sin\left(\frac{t}{N^{2i/d_{model}}}\right)$$

$PE(t, 2i+1) :$

$$\cos\left(\frac{t}{N^{2i/d_{model}}}\right)$$

$L = 512$

$t = \{0, 1, \dots, L\}$

$N = 10000$

$d_{model} = 14$

$i = \{0, 1, \dots, 6\}$

Reading the Formula as Oscillators

Rewrite the formula as:

$$PE(pos, 2i) = \sin(\omega_i pos),$$

$$PE(pos, 2i + 1) = \cos(\omega_i pos),$$

where

$$\omega_i = \frac{1}{10000^{2i/d_{\text{model}}}}.$$

Interpretation

Each sine–cosine pair is an oscillator running at frequency ω_i .

As i increases, ω_i decreases. Therefore:

- ▶ early dimensions change quickly;
- ▶ later dimensions change slowly.

Why the Number 10000?

The constant 10000 controls the range of wavelengths. The denominator

$$10000^{2i/d_{\text{model}}}$$

creates a geometric progression across dimensions.

Intuition

The model receives positional clocks at many different scales.

This resembles binary:

$$1, 2, 4, 8, 16, \dots$$

but uses smooth waves rather than abrupt bit flips.

Important

The exact constant is an engineering choice. The essential idea is the geometric spread of frequencies.

A Small Example

Suppose $d_{\text{model}} = 4$. Then:

$$PE(pos) = \left[\sin(pos), \cos(pos), \sin\left(\frac{pos}{100}\right), \cos\left(\frac{pos}{100}\right) \right].$$

For the first few positions:

pos	PE_0	PE_1	PE_2	PE_3
0	0.00	1.00	0.00	1.00
1	0.84	0.54	0.01	1.00
2	0.91	-0.42	0.02	1.00
3	0.14	-0.99	0.03	1.00

Reading the table

The first pair changes quickly; the second pair changes slowly.

Adding the Encoding to the Token Embedding

The final input to the Transformer is:

$$z_{pos} = e_{\text{token}(pos)} + PE(pos).$$

Example

For the sentence “**the bank raised rates...** ”, the token “bank” at position 1 is represented as:

$$z_1 = e_{\text{bank}} + PE(1).$$

If the same word appears in another position:

$$z_5 = e_{\text{bank}} + PE(5).$$

Consequence

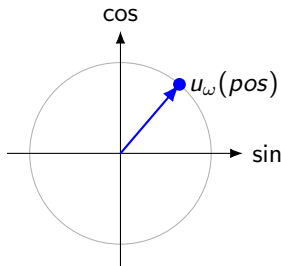
The same word obtains different input representations at different positions.

Why Sine and Cosine Together?

A Sine–Cosine Pair Is a Clock Hand. For one frequency ω , define:

$$u_{\omega}(pos) = \begin{bmatrix} \sin(\omega pos) \\ \cos(\omega pos) \end{bmatrix}.$$

This vector lies on the unit circle.



Increasing the position rotates the clock hand.

Relative Position as Rotation

Moving from pos to $pos + k$ rotates the sine–cosine pair.

Using trigonometric identities:

$$\sin(\omega(pos + k)) = \sin(\omega pos) \cos(\omega k) + \cos(\omega pos) \sin(\omega k),$$

$$\cos(\omega(pos + k)) = \cos(\omega pos) \cos(\omega k) - \sin(\omega pos) \sin(\omega k).$$

Key property

For a fixed offset k , the encoding at $pos + k$ can be obtained from the encoding at pos through a linear transformation.

The Rotation Matrix

The previous identities can be written as:

$$\begin{bmatrix} \sin(\omega(pos + k)) \\ \cos(\omega(pos + k)) \end{bmatrix} = \begin{bmatrix} \cos(\omega k) & \sin(\omega k) \\ -\sin(\omega k) & \cos(\omega k) \end{bmatrix} \begin{bmatrix} \sin(\omega pos) \\ \cos(\omega pos) \end{bmatrix}.$$

Interpretation

A relative shift in position corresponds to a rotation.

Why this matters

The transformation depends on the distance k , not on the absolute position pos .

Dot Products Reveal Relative Distance

For one sine–cosine clock:

$$u_{\omega}(pos) = \begin{bmatrix} \sin(\omega pos) \\ \cos(\omega pos) \end{bmatrix}.$$

The dot product between two positions is:

$$u_{\omega}(pos)^{\top} u_{\omega}(pos + k) = \cos(\omega k).$$

Important consequence

The similarity between two positional encodings depends on their relative distance.

Self-attention is based on dot products, so this property is especially useful.

The Transformer Architecture

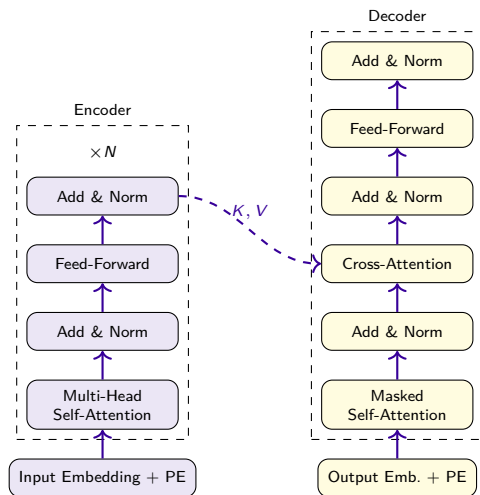
The Transformer: Overview (Vaswani et al., 2017)

The Transformer replaces recurrence entirely with attention. It consists of:

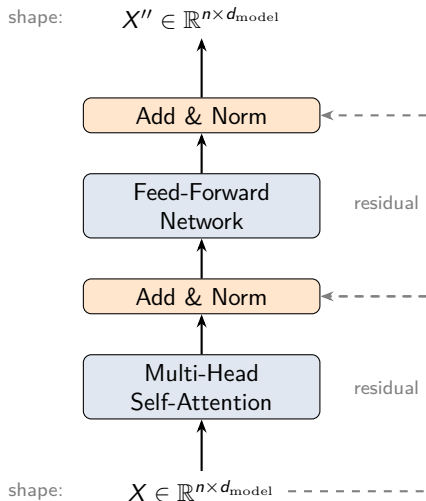
Encoder (left): processes the full input bidirectionally. Each layer contains:

- 1 Multi-head self-attention
- 2 Position-wise feed-forward network
- 3 Residual connections + layer normalisation

Decoder (right): generates output auto-regressively. Adds masked self-attention and cross-attention to encoder output.



One Encoder Block: Data Flow and Dimensions



Every encoder block applies four operations in sequence:

- 1 Multi-head self-attention**
Each token attends to every other token.
Input and output both live in $\mathbb{R}^{n \times d_{\text{model}}}$.
- 2 Add & Norm (first)**
The input X is added to the attention output (residual connection), then LayerNorm is applied token-wise: $X' = \text{LN}(X + \tilde{X})$.
- 3 Position-wise feed-forward network**
The same two-layer MLP is applied to each of the n token vectors independently.
- 4 Add & Norm (second)**
 $X'' = \text{LN}(X' + \text{FFN}(X'))$.

Stacking N Encoder Blocks: What Depth Achieves

Architecture: the output of block ℓ is the input of block $\ell + 1$:

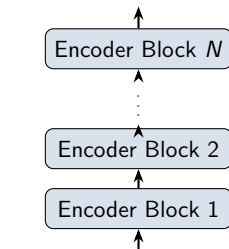
$$X^{(\ell)} = \text{EncoderBlock}\left(X^{(\ell-1)}\right), \quad \ell = 1, \dots, N.$$

All blocks share the *same structure* but have *independent, learned parameters*.

What successive layers tend to capture:

Layers	Representation content
Early	Surface form, local syntax, morphology
Middle	Phrase structure, entity type, coreference
Late	Semantic roles, long-range dependencies, task cues

$X^{(N)}$ — contextual representations



$X^{(0)}$ — embeddings + PE

Analogy

Stacking Transformer blocks resembles stacking convolutional layers in a CNN: each layer sees a progressively richer, more global view of the input.

Residual Connections and Layer Normalisation

Every sub-layer (attention or feed-forward) is wrapped in:

$$\text{output} = \text{LayerNorm}(x + \text{SubLayer}(x)).$$

Residual connections (He et al., 2016)

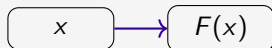
- ▶ Allow gradients to flow directly through the network via the identity shortcut.
- ▶ Enable training of very deep models: 6, 12, 24, 96+ layers.
- ▶ Without them, deep Transformers simply do not converge.

Layer normalisation (Ba et al., 2016)

- ▶ Normalises activations across features, not across the batch unlike BatchNorm.
- ▶ Stabilises training and reduces sensitivity to learning rate.
- ▶ Applied per-token, per-layer.

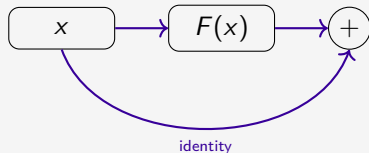
Residual Connections – Why They Work

Without residual



Gradient must flow through every layer.

With residual



Gradient has a direct path → deep networks can train.

The gradient of the output with respect to the input includes a term that is exactly **1** (the identity), regardless of what F does. This prevents gradient vanishing even with 96+ layers.

Why a Feed-Forward Layer after Self-Attention?

Self-attention allows each token to collect information from the other tokens in the sequence:

$$x_i \longrightarrow \tilde{x}_i,$$

where $\tilde{x}_i$ is a contextualized representation of token i .

However, attention mainly performs a *content-dependent mixing* of token representations. After this mixing step, the model still needs a mechanism to transform each contextualized vector into a richer feature representation.

Key idea

Self-attention decides *which* tokens interact; the feed-forward layer decides *how* each resulting token representation is transformed.

The Position-Wise Feed-Forward Network

In each Transformer block, the feed-forward sublayer is applied independently to each token position:

$$\text{FFN}(x_i) = W_2 \sigma(W_1 x_i + b_1) + b_2, \quad i = 1, \dots, n.$$

The same function is reused for all positions:

$$\text{FFN}(x_1), \text{FFN}(x_2), \dots, \text{FFN}(x_n).$$

If the input sequence is represented as a matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$, then the feed-forward layer maps

$$X \longrightarrow \text{FFN}(X) \in \mathbb{R}^{n \times d_{\text{model}}}.$$

Why Two Linear Layers?

The Transformer feed-forward sublayer is usually a two-layer neural network:

$$\text{FFN}(x_i) = W_2 \sigma(W_1 x_i + b_1) + b_2.$$

The first projection **expands** the representation:

$$W_1 : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{\text{ff}}}.$$

The second projection maps it **back** to the model dimension:

$$W_2 : \mathbb{R}^{d_{\text{ff}}} \rightarrow \mathbb{R}^{d_{\text{model}}}.$$

Typically, $d_{\text{ff}} > d_{\text{model}}$.

Why Two Linear Layers?

Interpretation

The expansion creates a higher-dimensional feature space; the non-linearity allows richer transformations; the final projection restores the dimension needed by the next Transformer block.

Self-Attention and Feed-Forward Layers Play Different Roles

Self-attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- ▶ mixes information across tokens;
- ▶ builds context-dependent representations;
- ▶ captures relations such as syntactic or semantic dependencies.

Feed-forward network

$$\text{FFN}(x_i) = W_2 \sigma(W_1 x_i + b_1) + b_2$$

- ▶ acts independently on each token;
- ▶ introduces non-linear feature transformation;
- ▶ refines the contextual representation produced by attention.

The Original Transformer: Key Hyperparameters

Vaswani et al. (2017) trained two model sizes on the WMT 2014 translation task. Understanding these numbers anchors the later discussion of scaling.

Hyperparameter	Base model	Large model
Encoder/decoder layers N	6	6
Model dimension d_{model}	512	1 024
Feed-forward dimension d_{ff}	2 048	4 096
Number of heads h	8	16
Head dimension $d_k = d_v$	64	64
Trainable parameters	≈ 65 M	≈ 213 M
Training steps	100 K	300 K
BLEU (EN $\rightarrow$ DE)	27.3	28.4

The Original Transformer: Key Hyperparameters

Note on head dimension

$d_k = d_{\text{model}}/h$: each head operates in a *lower-dimensional* space, so the total cost of multi-head attention is comparable to single-head attention at full dimension.

Perspective

65 M parameters was state-of-the-art in 2017. Contemporary LLMs range from 7 B (LLaMA 3.1 8B) to hundreds of billions of parameters—the architecture is the same; the scale is not.

Masked Self-Attention: Enabling Auto-regressive Generation

In the decoder, the model generates tokens one at a time, left to right. During training, it must not see future tokens – this would be “cheating”.

Solution: apply a mask to the attention score matrix before softmax:

$$s'_{ij} = \begin{cases} \frac{q_i^\top k_j}{\sqrt{d_k}} & \text{if } j \leq i, \\ -\infty & \text{if } j > i. \end{cases}$$

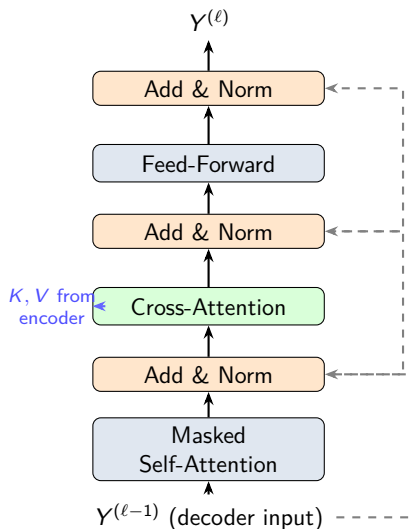
After softmax, $-\infty$ entries become 0 – future positions are invisible.

$$\begin{pmatrix} s_{11} & -\infty & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty \\ s_{31} & s_{32} & s_{33} & -\infty \\ s_{41} & s_{42} & s_{43} & s_{44} \end{pmatrix}$$

Causal mask: each token can only attend to itself and to previous tokens.

This is what makes GPT-style models auto-regressive: position i only “sees” positions $1, \dots, i$.

The Full Decoder Block: Three Sub-Layers



The decoder block has **three** sub-layers (vs. two in the encoder).

Sub-layer 1 — Masked self-attention

The decoder attends to its own previously generated tokens. The causal mask prevents attending to future positions.

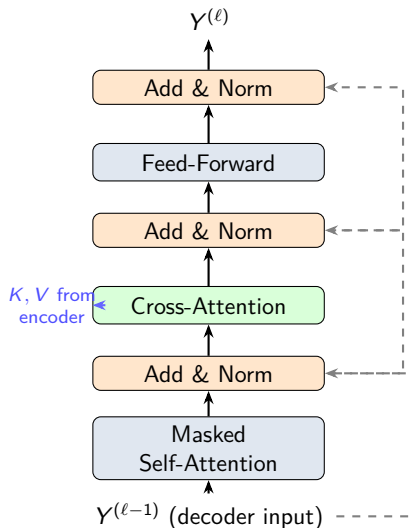
$$Q = K = V = Y^{(\ell-1)}, \quad \text{mask: } j \leq i \text{ only.}$$

Sub-layer 2 — Cross-attention

Queries come from the decoder; keys and values come from the encoder's final output $X^{(N)}$. This is how the decoder reads the source sequence.

$$Q = YW^Q, \quad K = X^{(N)}W^K, \quad V = X^{(N)}W^V.$$

The Full Decoder Block: Three Sub-Layers



Sub-layer 3 — Feed-forward network

Identical to the encoder FFN; applied position-wise.

Decoder-only models

GPT, LLaMA, Claude, Mistral: **no encoder, no cross-attention**. Sub-layer 2 is removed entirely. The model reads and generates within a single masked-self-attention + FFN stack.

Cross-Attention: Connecting Encoder and Decoder

In the original encoder-decoder Transformer used for translation and summarisation:

- ▶ The encoder processes the input sequence and produces contextual representations.
- ▶ The decoder generates the output sequence one token at a time.
- ▶ Cross-attention lets the decoder “look at” the encoder’s output:

$$Q = H_{\text{decoder}} W^Q, \quad K = H_{\text{encoder}} W^K, \quad V = H_{\text{encoder}} W^V.$$

Queries come from the decoder; keys and values come from the encoder.

Cross-Attention: Connecting Encoder and Decoder

Modern landscape

Most current LLMs such as GPT-style, Claude-style, LLaMA-style and Mistral-style models are **decoder-only** – they do not have an encoder or cross-attention. The entire architecture is masked self-attention + FFN, repeated N times.

Encoder-decoder models such as T5 and BART are still used for specific tasks like summarisation and translation.

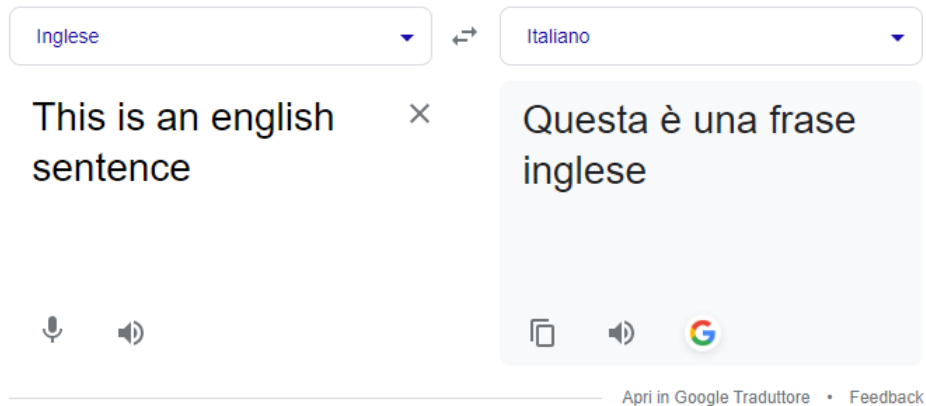
Why “Attention Is All You Need”?

	RNN / LSTM	CNN	Transformer
Sequential operations	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Maximum path length	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Parallelisable?	No	Yes	Yes
Complexity / layer	$\mathcal{O}(n \cdot d^2)$	$\mathcal{O}(k \cdot n \cdot d^2)$	$\mathcal{O}(n^2 \cdot d)$

The Transformer connects any two positions in the sequence with a single computation step (path length $\mathcal{O}(1)$), while being fully parallelisable on modern hardware such as GPUs and TPUs.

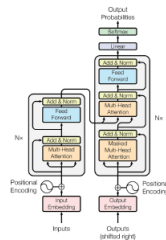
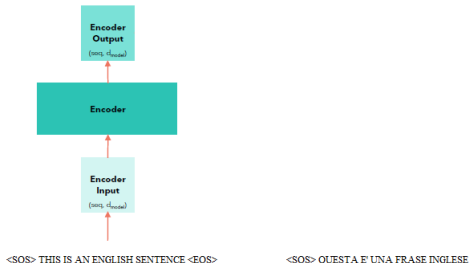
Transformed Training Vs Inference (Simplified!

Training Example



Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

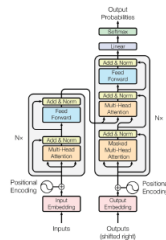
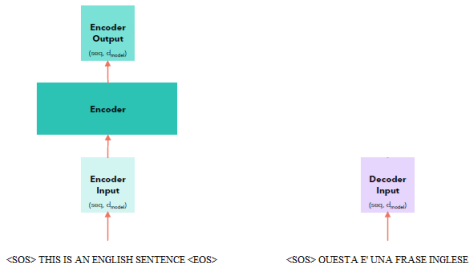


We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/0hkgproj/transformer-from-scratch-notes>

Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

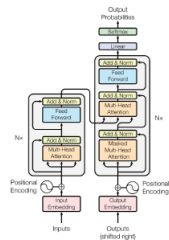
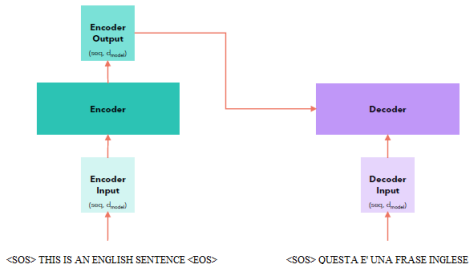


We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/0hkgproj/transformer-from-scratch-notes>

Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

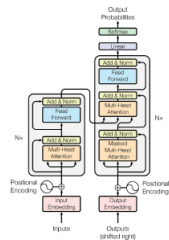
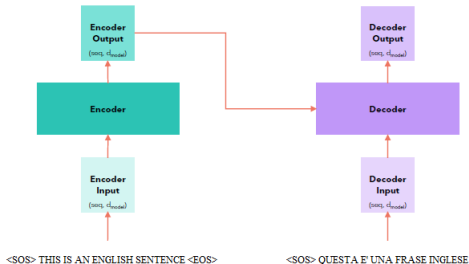


We prepend the **<SOS>** token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/uhkproj/transformer-from-scratch-notes>

Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

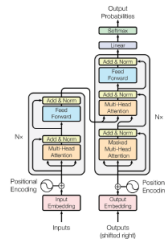
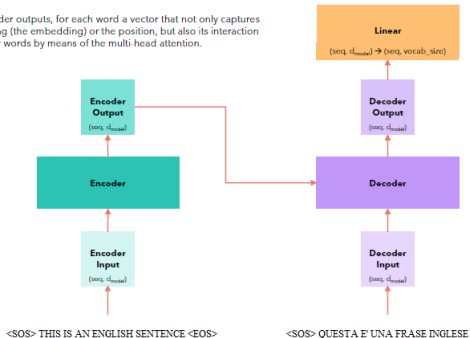


We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/uhkproj/transformer-from-scratch-notes>

Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

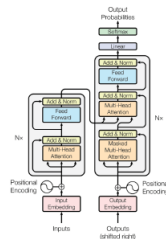
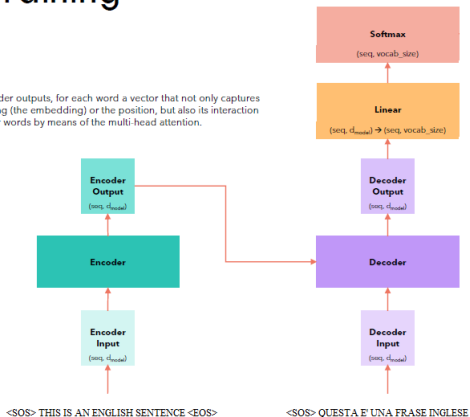


We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/uhkproj/transformer-from-scratch-notes>

Training

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.



We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/uhkproj/transformer-from-scratch-notes>

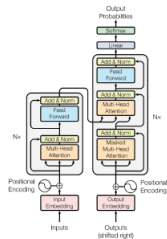
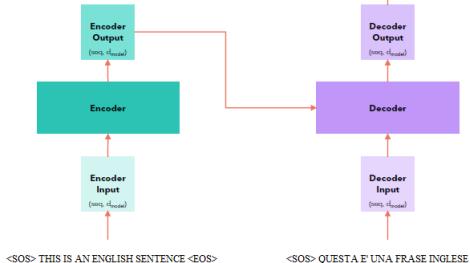
Training Example

Training

Time Step = 1

It all happens in one time step!

The encoder outputs, for each word a vector that not only captures its meaning (the embedding) or the position, but also its interaction with other words by means of the multi-head attention.

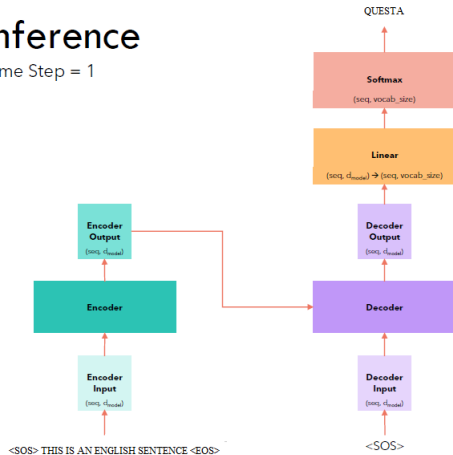


We prepend the <SOS> token at the beginning. That's why the paper says that the decoder input is shifted right.

Umar Jamil - <https://github.com/uhkproj/transformer-from-scratch-notes>

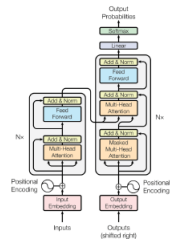
Inference

Time Step = 1



We select a token from the vocabulary corresponding to the position of the token with the maximum value.

The output of the last layer is commonly known as **logits**

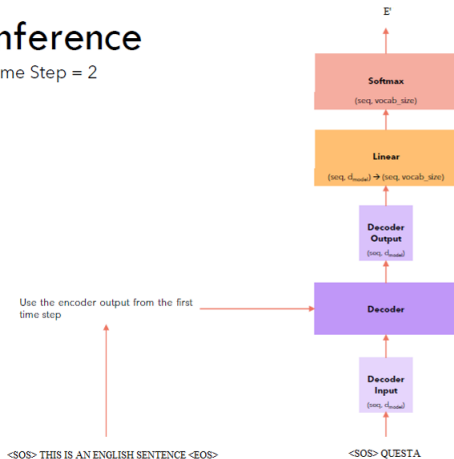


* Both sequences will have same length thanks to padding

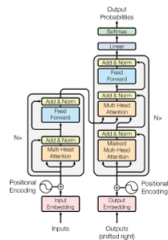
Umar Jamil - <https://github.com/hkproji/transformer-from-scratch-notes>

Inference

Time Step = 2



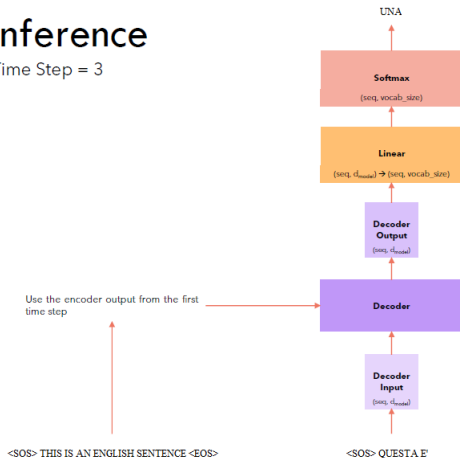
Since decoder input now contains **two** tokens, we select the softmax corresponding to the second token.



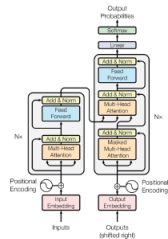
Append the previously output word to the decoder input

Inference

Time Step = 3



Since decoder input now contains **three** tokens, we select the softmax corresponding to the third token.

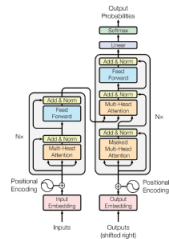
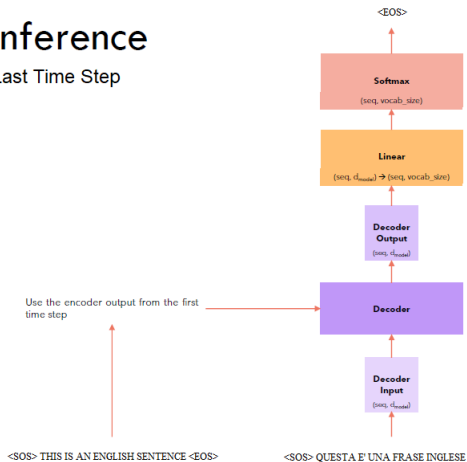


Append the previously output word to the decoder input

Umar Jamil - <https://github.com/hkproj/transformer-from-scratch-notes>

Inference

Last Time Step



Append the previously output word to the decoder input

Umar Jamil - <https://github.com/hkproj/transformer-from-scratch-notes>

From the Transformer to Specialised Architectures

The Original Task: Sequence-to-Sequence

The Transformer was introduced in *Attention Is All You Need* (Vaswani et al., 2017) to solve **machine translation**.

The core challenge

Given a source sentence of length n , produce a target sentence of length m , where $n \neq m$ in general.

Example: *“The Fed raised rates”* ($n=4$) $\longrightarrow$ *“La Fed ha alzato i tassi”* ($m=6$)

This single task actually hides **two distinct sub-problems**:

- 1 **Understanding the source:** build a rich internal representation that captures meaning, context, and long-range dependencies.
- 2 **Generating the target:** produce tokens *one at a time*, each conditioned on all tokens already generated *and* on the source representation.

The Original Task: Sequence-to-Sequence

Key insight

The Transformer solves these two sub-problems with **two structurally different components**: the **encoder** and the **decoder**.

Two Roles Within One Architecture

The Encoder

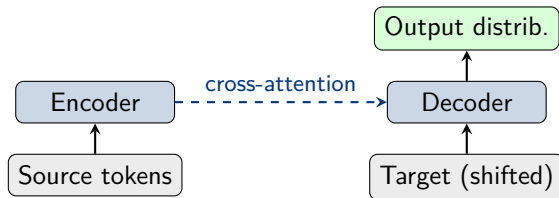
- ▶ Reads the **entire** source in parallel.
- ▶ N layers of self-attention + feed-forward.
- ▶ Output: $\mathbf{Z} \in \mathbb{R}^{n \times d}$, one contextual embedding per token.
- ▶ All positions attend freely to each other.

Role: build a *memory* of the source.

The Decoder

- ▶ Generates the target **one token at a time**.
- ▶ *Masked* self-attention: pos. t only attends to $1, \dots, t-1$.
- ▶ Reads encoder memory via **cross-attention**.
- ▶ Outputs a probability over the vocabulary.

Role: generate conditioned on the memory.



Key observation

Cross-attention is the *only* coupling between encoder and decoder. Remove it and the two components become **fully independent**.

Cross-Attention: The Bridge Between Encoder and Decoder

Inside every decoder layer there is a **cross-attention** sub-layer in addition to the usual masked self-attention.

Cross-attention formula

$$\text{CrossAttn}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^{\top}}{\sqrt{d_k}}\right) V_{\text{enc}}$$

- ▶ **Queries** Q_{dec} : from the decoder's hidden states at the current generation step.
- ▶ **Keys and Values** $K_{\text{enc}}, V_{\text{enc}}$: from the encoder output $\mathbf{Z}$ — computed once, reused at every decoder step.

Cross-Attention: The Bridge Between Encoder and Decoder

Intuition

At each step the decoder *asks a question* (query) about what it needs next, and the encoder *answers* by highlighting the most relevant source tokens. This mirrors how a human translator alternates between reading the original and writing the translation.

Structural observation

Cross-attention is the *only* information pathway between encoder and decoder. Remove it and the two components become fully independent — each exploitable on its own.

A Key Observation: Two Independent Capabilities

Once we recognise what each component does *on its own*, a natural question arises: do we always need both?

The two standalone capabilities

- ▶ The **encoder alone** produces rich contextual representations of any text — no generation involved.
- ▶ The **decoder alone** can generate text autoregressively from a conditioning prefix — no external memory needed.
- ▶ The coupling via cross-attention is **task-specific**, not architecturally mandatory.

A Key Observation: Two Independent Capabilities

Two research trajectories

Encoder only Maximise representation quality for *understanding* tasks: classification, named-entity recognition, semantic similarity, retrieval.

Decoder only Maximise generation quality for *production* tasks: summarisation, dialogue, code generation, question answering.

These two paths produced two landmark families: **BERT** (encoder only) and **GPT / modern LLMs** (decoder only).

Encoder-Only Models: BERT

BERT (Devlin et al., 2018) — *Bidirectional Encoder Representations from Transformers* — is a pure encoder stack trained on two self-supervised objectives.

Architecture

- ▶ Encoder stack only; no decoder, no cross-attention.
- ▶ All tokens attend to **all other tokens** in both directions (*bidirectional*).
- ▶ Output: one contextual embedding $\mathbf{z}_i \in \mathbb{R}^d$ per input token.

Pre-training objectives

- 1 **Masked LM (MLM)**: randomly mask 15 % of tokens and predict them — forces rich contextual representations.
- 2 **Next Sentence Prediction (NSP)**: predict whether two sentences are consecutive.

Encoder-Only Models: BERT

Why bidirectionality matters

In “*The bank raised rates after the flood hit the river bank*”, resolving the two meanings of *bank* requires attending to tokens on **both sides** — impossible for a left-to-right causal model.

Downstream use

Add a small task-specific head on top of BERT embeddings and fine-tune: sentiment analysis, NER, question answering, semantic search — with minimal labelled data.

Decoder-Only Models: GPT and Modern LLMs

GPT (Radford et al., 2018) — *Generative Pre-trained Transformer* — uses a decoder stack without cross-attention, trained on one self-supervised objective: **next-token prediction**.

Architecture

- ▶ Decoder stack only; no encoder, no cross-attention.
- ▶ **Causal self-attention**: position t attends only to $1, \dots, t$.
- ▶ At each step: a probability over the vocabulary.

Pre-training objective

- ▶ $\mathcal{L} = -\sum_t \log P(x_t \mid x_1, \dots, x_{t-1})$
- ▶ Left-to-right factorisation of any text corpus.
- ▶ No human labels required.

Decoder-Only Models: GPT and Modern LLMs

From GPT to modern chatbots

Scaling (more layers, more data) produces *emergent capabilities*: in-context learning, chain-of-thought reasoning, instruction following. Adding **RLHF** aligns the generator with human intent — the recipe behind GPT-4, Claude, Gemini, and similar systems.

Critical difference from BERT

Representations are *unidirectional* (no future tokens visible) and less suited to tasks that benefit from full bidirectional context.

Summary: The Specialisation Landscape

	Full Transformer	Encoder-only	Decoder-only
Example	Vaswani et al. 2017	BERT (2018)	GPT series
Components	Enc + Dec	Encoder only	Decoder only
Attention type	Self + Cross	Bidirectional self	Causal self
Generation	seq-to-seq	×	autoregressive
Representations	source-conditioned	bidirectional	unidirectional
Primary use	translation, ASR	classif., NER, search	generation, chat

The big picture

The original Transformer coupled two powerful but *separable* ideas. Recognising this separation unlocked a productive specialisation:

- ▶ **Understanding tasks** → encoder-only (bidirectional context, rich embeddings → BERT family).
- ▶ **Generation tasks** → decoder-only (autoregressive, scalable to very large models → GPT family).
- ▶ **Seq-to-seq tasks** → full architecture (translation, abstractive summarisation, speech recognition).

Summary and Bridge

Key Takeaways

- 1 **Contextual embeddings** solve the polysemy problem: one word, different vectors depending on context.
- 2 **Attention** is a differentiable soft lookup: query-key similarity determines how much each value contributes.
- 3 **Self-attention** lets every token attend to every other token in a single parallel step, replacing sequential processing.
- 4 Contextualised representations emerge because semantically related tokens exchange more information via higher attention weights.
- 5 **Multi-head attention** decomposes contextual relevance across independent subspaces – like multi-factor decomposition in finance.
- 6 **Positional encoding** injects sequence order into a permutation-equivariant architecture.
- 7 The Transformer combines attention, FFN, residual connections, and normalisation into a scalable, parallelisable architecture.
- 8 The $\mathcal{O}(n^2)$ cost in sequence length is the key bottleneck, motivating chunking, retrieval, and efficient attention methods.

Pre-trained Models, LLMs, and Financial Applications

Now that we understand how the Transformer works, Lesson 4 will address what happens when we scale it:

- ▶ **BERT and encoder models:** bidirectional pre-training, masked language modelling, fine-tuning for financial NLP.
- ▶ **GPT and decoder models:** autoregressive generation, in-context learning, prompting.
- ▶ Scaling laws, emergent behaviour, and healthy scepticism.
- ▶ Alignment: RLHF, DPO, and why it matters for finance.
- ▶ Financial applications: where LLMs add value, where caution is essential, and the discipline of always benchmarking against simpler alternatives.
- ▶ Temperature, sampling, and decoding strategies: how to control generation in practice.